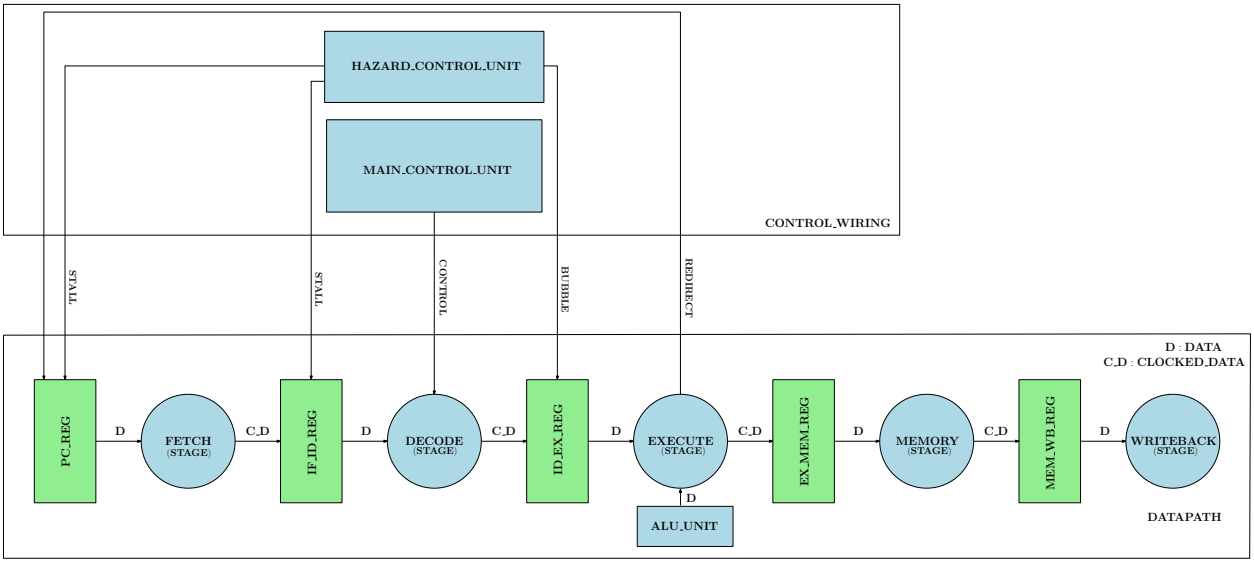
CS6600 PA-1 Report

Team-6: EE25M067, EE25M102, EE25M011

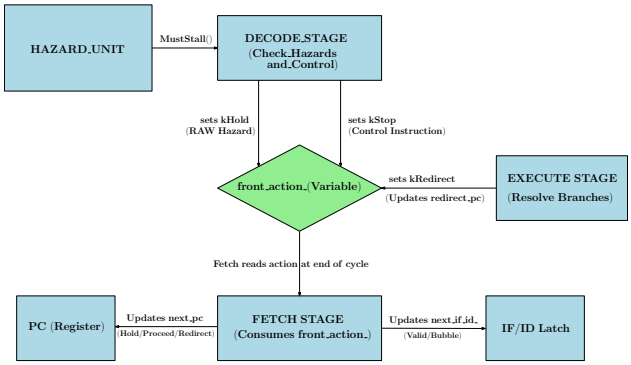
Datapath and control units:



1. **BuildDesign()**: We instantiated the 5-stage pipeline, linking registers to stages via kData edges and stages to registers via kClockedData edges. The functional units were registered and wired strictly to specification: the ALU and Main Control feed the execution and decode stages, the Hazard unit routes stalls and bubbles to the pc, if/id, id/ex registers, and the Execute stage is wired to redirect the PC.
2. BEQ, BNE, J, HALT are considered as control instructions. Except J, HALT, NOP all instructions use rs1 field. Except ADDI, LW, J, HALT, NOP all instructions use rs2 field.
3. Completed **MainControl::Decode()** function mapping every opcode to its required datapath signals by correctly asserting reg_write, mem_read, mem_write, use_immediate, halt, and the specific alu_op for the execution and memory stages to consume. Control.use_immediate = true lets the alu choose immediate field as its second operand instead of rs2.
4. **Alu::Run**: Appropriate operations performed as per the operation.
5. **Core::Decode()**: The Decode stage's strict execution order is essential for pipeline synchronization. First, data hazards are evaluated; detecting one triggers a kHold signal and an early return, which stalls the front-end and naturally bubbles the ID/EX latch. Second, hazard-free instructions safely read architectural registers and populate ID/EX. Finally, control instructions are evaluated last. This guarantees that dependent branches stall correctly before advancing to Execute, and only then issue a kStop to halt the Fetch stage.
6. **Core::Execute()**: Alu operation is performed with operands chosen based on control signals. The stage evaluates branch (BEQ, BNE) and jump (J) instructions. If a branch condition is met or a jump is evaluated, the stage calls the Redirect() function to set front_action_ = FrontAction::kRedirect and records the target address.
7. **HazardControl**: (discussed below)
8. **Core::Memory()**: if control.mem_read is true, next_mem_wb_.result is updated with the word at ex_mem_.result

Hazard Detection and Stalling Hazard detection is handled by the HazardControl unit during the Decode stage, specifically targeting Read-After-Write (RAW) data dependencies since our design does not implement data forwarding. For every valid instruction entering Decode, the unit first determines if the instruction actually reads from source registers rs1 or rs2 based on its opcode. If it does, HazardControl::MustStall() uses a helper function to compare these source registers against the destination registers (rd) of older instructions

currently in the ID/EX and EX/MEM latches. A data hazard is flagged if a match is found on a non-zero register, provided the older instruction is valid and its control signals intend to write back to the register (reg_write is true). When a hazard is flagged, the Decode stage sets the front_action_ to kHold and returning immediately. This early return leaves the next_id_ex_ latch invalid, injecting a bubble into the Execution stage. Meanwhile, the Fetch stage reads the kHold signal to freeze the Program Counter and retain the current IF/ID latch, the dependent instruction is re-evaluated in the next cycle.



Branch and Control Resolution Branch resolution is divided between early detection in the Decode stage and actual evaluation in the Execute stage, with no branch prediction used. When the Decode stage identifies a control instruction (like BEQ, BNE, J, or HALT) using the IsControl() function, it allows the instruction to proceed into the ID/EX latch but sets front_action_ = FrontAction::kStop. This signals the Fetch stage to freeze the PC and insert a bubble into the IF/ID latch, preventing any new instructions from being fetched while the branch is unresolved. One cycle later, the control instruction enters the Execute stage, where the target address and branch conditions are evaluated (e.g., comparing rs1 and rs2 for equality). If a branch is taken or a jump is executed, the Execute stage triggers a Redirect(), updating front_action_ to kRedirect and providing the new target PC. The Fetch stage processes this redirection at the end of the cycle by updating the PC to the new address and flushing the fetch latch.

For example.pisa:

Cycles = 20; Retired = 8; Data Stall = 6; Control Stalls = 2.

The program example.pisa has 10 instructions but BEQ evaluates to true (12 == 12) and takes the branch to 0x20. The two instructions between 0x18 and 0x1c are skipped. Hence, there are 8 retired instructions. ADD 3 1 2 0 depends on x2 from the immediately preceding ADDI instruction. It stalls for 2 cycles. SW 0 0 3 0 depends on x3 from the immediately preceding ADD instruction. It stalls for 2 cycles. BEQ 0 4 3 0x20 depends on x4 from the immediately preceding LW instruction. It stalls for 2 cycles. Hence, in total 6 data stalls. BEQ and HALT contribute for the 2 control stalls. The 20 total cycles consist of 8 cycles to issue the retired instructions, 4 cycles to drain the final HALT through the remaining pipeline stages, 6 cycles of data stalls, 1 cycle lost to the BEQ redirect flush, and 1 cycle lost to the HALT control stall bubble.

For custom.pisa:

Cycles = 27; Retired = 11; Data Stalls = 6; Control Stalls = 4.

The custom program contains multiple failure blocks, but the not-taken BEQ, the unconditional J, and the taken BNE correctly skip these trapped instructions, Hence, 11 retired instructions. ADD 3 1 2 0 depends on x2 from the immediately preceding ADDI instruction, stalling for 2 cycles. SW 0 0 3 0 depends on x3 from the immediately preceding ADD instruction, stalling for 2 cycles. ADD 5 4 4 0 depends on x4 from the immediately preceding LW instruction, stalling for 2 cycles. Hence, 6 data stalls in total. The BEQ, J, BNE, and HALT instructions contribute to the 4 control stalls. The 27 total cycles consist of 11 cycles to issue the retired instructions, 4 cycles to drain the final HALT through the remaining pipeline stages, 6 cycles of data stalls, and 6 cycles lost to the redirect flushes and control stall bubbles generated by the branch, jump, and halt instructions

```
PIPEISA 1
ENTRY 0x00000000

INST 0x00000000 ADDI 1 0 0 10
INST 0x00000004 ADDI 2 0 0 20
INST 0x00000008 ADD 3 1 2 0
INST 0x0000000c SW 0 0 3 0
INST 0x00000010 LW 4 0 0 0
INST 0x00000014 ADD 5 4 4 0
INST 0x00000018 BEQ 0 1 2 0x00000030
INST 0x0000001c J 0 0 0 0x00000028
INST 0x00000020 ADDI 7 0 0 999
INST 0x00000024 HALT 0 0 0 0
INST 0x00000028 BNE 0 1 2 0x00000034
INST 0x0000002c ADDI 7 0 0 888
INST 0x00000030 HALT 0 0 0 0
INST 0x00000034 ADDI 7 0 0 777
INST 0x00000038 HALT 0 0 0 0

END
```